

PRÁCTICA 3:

FRONTEND



- Juan Manuel Valenzuela González
- Artur Vargas Carrión
- Jorge Repullo Serrano
- Eduardo González Bautista
- Rubén Oliva Zamora
- David Muñoz del Valle

- amcgil@uma.es
- arturvargas797@uma.es
- jorgers4@uma.es
- edugb@uma.es
- rubenoliva@uma.es
- davidmunvalle@uma.es

ÍNDICE DE CONTENIDO

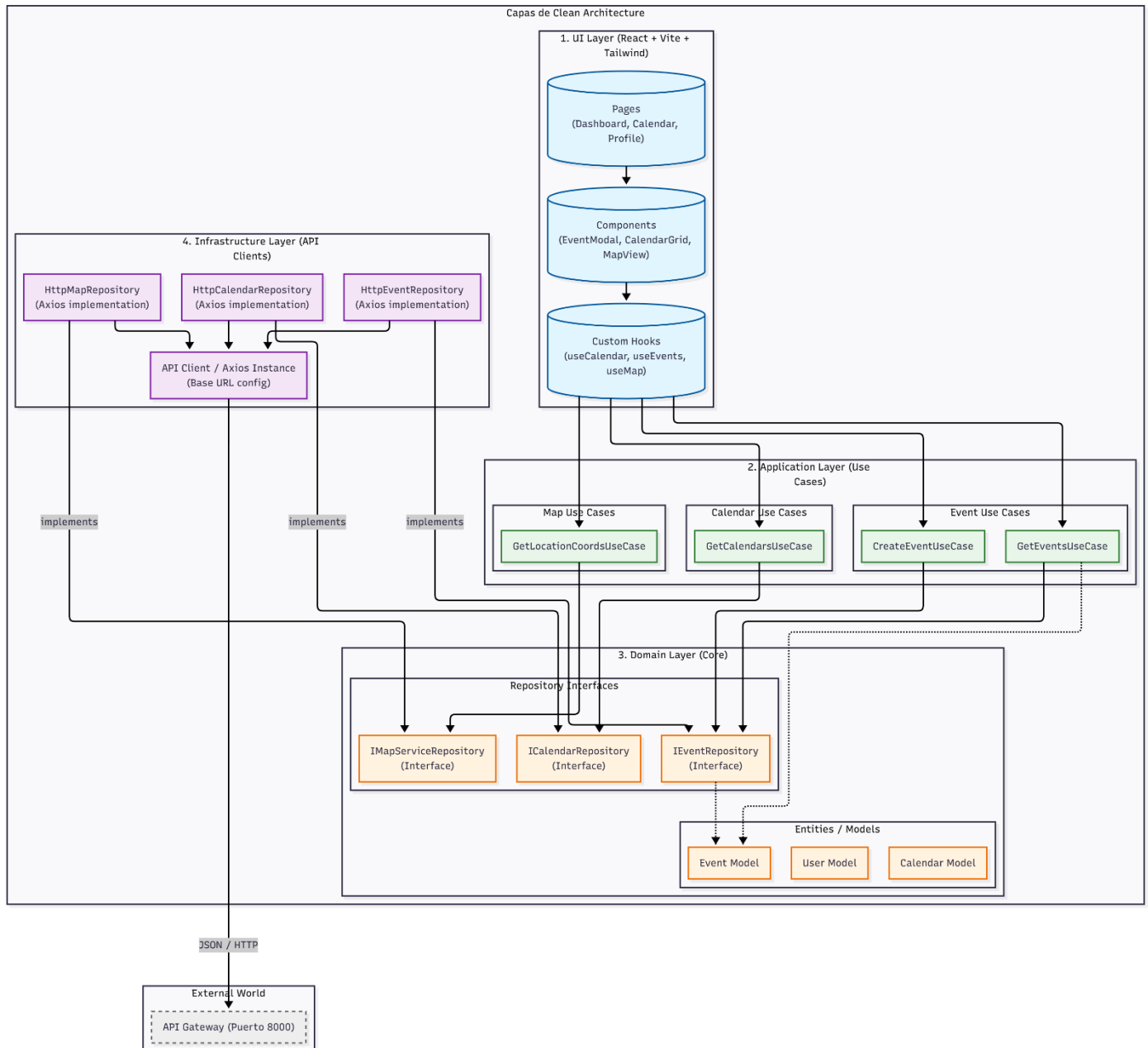
1. CAMBIOS EN LA ARQUITECTURA	4
1.1. ARQUITECTURA DEL FRONTEND	4
1.1.1. <i>Visión general</i>	4
1.1.2. <i>Estructura de capas</i>	5
1.1.2.1. Capa de presentación (UI layer)	5
1.1.2.2. Capa de aplicación (application layer)	5
1.1.2.3. Capa de dominio (domain layer)	5
1.1.2.4. Capa de infraestructura (infrastructure layer)	5
1.1.3. <i>Flujo de datos e interacción</i>	5
1.1.4. <i>Justificación del diseño</i>	6
2. REQUISITOS CONSIDERADOS	7
2.1. REQUISITOS TÉCNICOS Y DE IMPLEMENTACIÓN	7
2.1.1. <i>Conexión a backend</i>	7
2.1.2. <i>Versionado de API y base de datos</i>	7
2.2. REQUISITOS DE FUNCIONALIDAD	7
2.2.1. <i>Gestión de calendarios y eventos</i>	7
2.2.2. <i>Contenido de eventos</i>	7
2.2.3. <i>Funcionalidad de búsqueda</i>	7
2.3. REQUISITOS DE INTEGRACIÓN ESPECÍFICA	7
2.3.1. <i>Visualización de mapas</i>	7
2.3.2. <i>Visualización y almacenamiento de imágenes en la nube</i>	8
2.3.3. <i>Comentarios y notificaciones</i>	8
3. TECNOLOGÍAS USADAS	9
3.1. BACKEND	9
3.1.1. <i>Lenguaje</i>	9
3.1.2. <i>Framework</i>	9
3.1.3. <i>Contenedores</i>	9
3.1.4. <i>Gestión de paquetes</i>	9
3.1.5. <i>Acceso a APIs externas</i>	9
3.1.6. <i>Base de datos</i>	9
3.2. FRONTEND	9
3.2.1. <i>Lenguaje</i>	9
3.2.2. <i>Framework</i>	9
3.2.3. <i>Build / bundler</i>	10
3.2.4. <i>Estilos</i>	10
3.2.5. <i>Cliente HTTP</i>	10
3.2.6. <i>Gestión de paquetes</i>	10
3.3. INTEGRACIONES EXTERNAS	10
3.3.1. <i>Almacenamiento</i>	10
3.3.2. <i>Calendarios</i>	10
3.3.3. <i>Geolocalización</i>	10
3.3.4. <i>Correo electrónico</i>	11
4. BASE DE DATOS	12
4.1. CAMBIOS EN EL DISEÑO	12
4.1.1. <i>Nueva colección: caché de geocodificación</i>	12
4.1.2. <i>Ampliación de documentos existentes</i>	12

4.2.	INSTRUCCIONES DE ACCESO	13
5.	MICROSERVICIOS	14
5.1.	VERSIONADO DE LOS MICROSERVICIOS.....	14
5.1.1.	<i>Arquitectura del versionado</i>	14
5.1.1.1.	Organización del sistema	14
5.1.1.2.	Implementación de rutas	14
5.1.2.	<i>Patrón de diseño: Abstract Factory</i>	14
5.1.3.	<i>Filosofía de desarrollo incremental</i>	14
5.2.	CAMBIOS EN LOS MICROSERVICIOS	15
5.2.1.	<i>Calendar Service</i>	15
5.2.1.1.	GET /v2/calendars – Obtener todos los calendarios.....	15
5.2.1.2.	DELETE /v2/calendars/{calendar_id}/recursive – Eliminar calendario recursivamente	15
5.2.1.3.	GET /v2/calendars/search/by-text – Buscar por texto.....	15
5.2.1.4.	GET /v2/calendars/search/by-creator-name – Buscar por nombre del creador.....	15
5.2.1.5.	POST /v2/calendars/{calendar_id}/comments – Agregar comentario a un calendario	15
5.2.2.	<i>Event Service</i>	15
5.2.2.1.	GET /v2/events – Obtener todos los eventos	15
5.2.2.2.	DELETE /v2/events/by-calendar/{calendar_id} – Eliminar eventos de un calendario	16
5.2.2.3.	GET /v2/events/search/by-date-range – Buscar eventos por rango de fechas	16
5.2.2.4.	POST /v2/events/{event_id}/comments – Agregar comentario.....	16
5.2.2.5.	GET /v2/events/search/by-text - Búsqueda full-text en eventos	16
5.2.2.6.	GET /v2/events/search/by-calendar-title - Buscar eventos por título del calendario.....	16
5.2.2.7.	GET /v2/events/search/by-location - Buscar eventos por ubicación.....	16
5.2.2.8.	GET /v2/events/search/advanced - Búsqueda avanzada de eventos.....	17
5.2.3.	<i>Notification Service</i>	17
5.2.4.	<i>Integration Service</i>	17
5.2.4.1.	POST /v2/integrations/imports/google – Importar desde Google Calendar	17
5.2.4.2.	POST /v2/integrations/imports/teamup – Importar desde Teamup	17
5.2.4.3.	GET /v2/integrations/osm/geocode – Geocodificar dirección.....	17
5.2.4.4.	GET /v2/integrations/osm/reverse-geocode – Geocodificación inversa.....	17
5.2.4.5.	GET /v2/integrations/osm/search-places – Buscar lugares	18
5.2.4.6.	GET /v2/integrations/osm/cache/stats - Estadísticas del caché de geocodificación	18
5.2.4.7.	DELETE /v2/integrations/osm/cache - Limpiar caché de geocodificación	18
5.2.4.8.	POST /v2/integrations/s3/upload-direct – Subir imagen directa.....	18
5.2.4.9.	POST /v2/integrations/s3/upload – Obtener URL presigned.....	18
5.2.4.10.	GET /v2/integrations/s3/images - Listar imágenes.....	18
5.2.4.11.	GET /v2/integrations/s3/images/{image_key} - Obtener metadatos de imagen.....	19
5.2.4.12.	DELETE /v2/integrations/s3/images/{image_key} - Eliminar imagen.....	19
5.2.4.13.	GET /v2/integrations/s3/download/{image_key} - Obtener URL de descarga.....	19
5.2.4.14.	POST /v2/integrations/s3/images/bulk-delete - Eliminar múltiples imágenes	19
5.2.4.15.	GET /v2/integrations/s3/images/{image_key}/exists - Verificar si imagen existe.....	19
5.2.4.16.	POST /v2/integrations/email/send – Enviar correo	19
5.2.4.17.	POST /v2/integrations/email/send-digest – Enviar resumen diario	19
5.2.4.18.	POST /v2/integrations/email/send-comment-notification – Enviar notificación de comentario	20
5.2.4.19.	POST /v2/integrations/digest/send-all – Enviar digest masivo.....	20
5.2.4.20.	POST /v2/integrations/digest/send-user – Enviar digest a usuario	20
5.2.4.21.	GET /v2/integrations/digest/preview/{user_external_id} - Vista previa del resumen diario	20
5.2.5.	<i>User Service</i>	20
5.2.5.1.	GET /v2/users/by-external-id/{external_id} – Obtener por External ID	20
5.2.5.2.	GET /v2/users – Listar usuarios	20
5.2.5.3.	GET /v2/users/{user_id}/notification-preferences – Obtener preferencias	21
5.2.5.4.	PUT /v2/users/{user_id}/notification-preferences – Actualizar preferencias.....	21
5.2.5.5.	GET /v2/users/{user_id} – Obtener usuario por ID	21
5.2.5.6.	PUT /v2/users/{user_id} – Actualizar usuario	21

5.2.5.7.	POST /v2/users/seed-dev-users – Crear usuarios de prueba.....	21
6.	FRONTEND.....	22
6.1.	FUNCIONALIDAD.....	22
6.1.1.	<i>Gestión de sesión y usuarios.....</i>	22
6.1.2.	<i>Gestión de calendarios (Dashboard).....</i>	22
6.1.3.	<i>Gestión de eventos</i>	22
6.1.4.	<i>Búsqueda global y filtrado</i>	22
6.1.5.	<i>Aspectos técnicos de la interfaz (UI/UX)</i>	22
6.2.	DISEÑO.....	23
6.2.1.	<i>Accesibilidad.....</i>	23
6.2.1.1.	Diseño responsivo	23
6.2.2.	<i>Estilo</i>	24
6.2.2.1.	Características principales	24
6.2.2.2.	Paleta de colores.....	24
6.3.	INTEGRACIÓN DE SERVICIOS EXTERNOS	24
6.3.1.	<i>Visualización de mapas – OpenStreetMap.....</i>	24
6.3.1.1.	Relación con el servicio de integración.....	24
6.3.1.2.	Funcionalidades implementadas.....	25
6.3.2.	<i>Almacenamiento de imágenes – AWS S3.....</i>	25
6.3.2.1.	Relación con el servicio de integración.....	25
6.3.2.2.	Funcionalidades implementadas.....	25
6.3.3.	<i>Envío de emails – SendGrid.....</i>	26
6.3.3.1.	Relación con el servicio de integración.....	26
6.3.3.2.	Funcionalidades implementadas.....	26
6.3.4.	<i>Otros servicios</i>	26
6.3.4.1.	Importación de calendarios – TeamUp	26
6.3.4.2.	Importación de calendarios – Google Calendar.....	27
7.	INSTRUCCIONES DE DESPLIEGUE.....	28
7.1.	REQUISITOS PREVIOS.....	28
7.2.	CONFIGURACIÓN.....	28
7.3.	DESPLIEGUE EN DOCKER COMPOSE.....	28
7.4.	ACCESO AL SISTEMA.....	28

1. Cambios en la arquitectura

1.1. Arquitectura del frontend



1.1.1. Visión general

El desarrollo del frontend de **Basmati** sigue estrictamente los principios de **Clean Architecture**. Este patrón de diseño se ha elegido para garantizar la escalabilidad, mantenibilidad y testabilidad de la aplicación, desacoplando la interfaz de usuario (UI) de la lógica de negocio y los servicios externos.

La arquitectura se estructura en cuatro capas concéntricas con un flujo de dependencia unidireccional: las capas externas (UI e infraestructura) dependen de las capas internas (aplicación y dominio), pero **nunca** a la inversa.

1.1.2. Estructura de capas

1.1.2.1. Capa de presentación (UI layer)

Es la capa visible para el usuario. Su única responsabilidad es mostrar datos y capturar interacciones. La UI **nunca** importa ni utiliza los repositorios directamente. No contiene lógica de negocio compleja ni realiza llamadas directas a la API. Sus piezas clave son:

- **Pages:** vistas completas de la aplicación (ej. *Dashboard, Calendar, Profile*).
- **Components:** elementos reutilizables de la interfaz (ej. *EventModal, CalendarGrid, MapView*).
- **Custom Hooks:** actúan como el nexo entre la vista y la lógica de negocio. Los hooks (ej. *useCalendar, useEvents*) instancian y ejecutan los casos de uso.

1.1.2.2. Capa de aplicación (application layer)

Contiene la lógica de negocio pura de la aplicación, agnóstica de la interfaz (React) y de la fuente de datos (API).

- **Casos de uso (use cases):** clases o funciones que encapsulan una acción específica del negocio. Orquestan el flujo de datos hacia y desde la capa de dominio.
 - *Ejemplos:* `CreateEventUseCase`, `GetEventsUseCase`, `GetCalendarsUseCase`, `GetLocationCoordsUseCase`.
- **Responsabilidad:** recibir input de la UI, ejecutar reglas de negocio y llamar a las interfaces del repositorio para obtener o persistir datos.

1.1.2.3. Capa de dominio (domain layer)

Es el núcleo (Core) del sistema. Esta capa es totalmente independiente y no tiene dependencias externas (ni React, ni Axios, ni librerías de terceros).

- **Entidades / Modelos:** interfaces TypeScript que definen la estructura de los datos del negocio (ej. *EventModel, CalendarModel, UserModel*).
- **Interfaces de repositorio:** contratos abstractos que definen qué operaciones de datos son posibles, sin especificar cómo se realizan (ej. *IEventRepository, ICalendarRepository*).
- **Objetivo:** definir las reglas y tipos de datos que gobiernan la aplicación.

1.1.2.4. Capa de infraestructura (infrastructure layer)

Es la capa encargada de la comunicación con el mundo exterior (API Gateway). Implementa las interfaces definidas en la capa de dominio.

- **Implementación de repositorios:** clases concretas que utilizan un cliente HTTP para cumplir con los contratos del dominio.
 - *Ejemplos:* `HttpEventRepository`, `HttpMapRepository`. Estas clases realizan las llamadas reales (`axios.get`, `axios.post`).
- **API client:** configuración base del cliente HTTP (Axios) con interceptores, manejo de errores y configuración de URL base.
- **Mappers:** transformadores que adaptan el formato de datos de la API (JSON externo) al formato de los Modelos de Dominio internos.

1.1.3. Flujo de datos e interacción

El flujo de la información sigue un camino estricto para mantener el desacoplamiento:

1. **Interacción:** el usuario realiza una acción en un **Componente** (ej. hacer clic en "Ver eventos").
2. **Hook:** el componente delega la acción a un **Custom Hook** (`useEvents`).
3. **Caso de uso:** el hook invoca al **caso de uso** correspondiente (`GetEventsUseCase`).
4. **Dominio (abstracción):** el caso de uso llama a la función definida en la **interfaz del repositorio** (`IEventRepository`), sin saber quién la implementa.

5. **Infraestructura (implementación):** en tiempo de ejecución, se utiliza la implementación concreta (HttpEventRepository) que realiza la petición HTTP al **API Gateway** (Puerto 8000).
6. **Retorno:** los datos fluyen de vuelta hacia arriba, transformándose en modelos de dominio y actualizando el estado de React en la UI.

1.1.4. Justificación del diseño

- **Independencia de frameworks:** la lógica de negocio (capas 2 y 3) no depende de React. Si en el futuro se cambiara la librería de UI, la lógica de negocio permanecería intacta.
- **Testabilidad:** al depender de interfaces (Dominio) y no de implementaciones concretas (Axios), es trivial realizar pruebas unitarias simulando (mocking) los repositorios.
- **Estandarización:** el uso de snake_case en métodos y variables, junto con documentación estricta (Docstrings), asegura consistencia en el código base.
- **Escalabilidad:** la separación clara de responsabilidades permite que el equipo trabaje en paralelo en componentes visuales, lógica de negocio e integración de API sin conflictos.

2. Requisitos considerados

2.1. Requisitos técnicos y de implementación

2.1.1. Conexión a backend

El frontend debe consumir los **microservicios REST** de las entidades desplegados en la práctica anterior.

2.1.2. Versionado de API y base de datos

Para garantizar la **retrocompatibilidad**, cualquier modificación en el esquema de la base de datos o en la API REST debe asegurar el funcionamiento de las versiones anteriores. Para ello, se implementarán mecanismos de **versionado** tanto de datos como de servicios. Asimismo, toda actualización en la API REST deberá estar documentada mediante la especificación **OpenAPI**.

2.2. Requisitos de funcionalidad

2.2.1. Gestión de calendarios y eventos

El sistema debe proporcionar herramientas completas para la organización de tiempo y actividades:

- **Gestión integral (CRUD)**: el sistema permitirá la **creación, consulta, edición y eliminación** de calendarios y eventos. No obstante, se aplicará una **política de permisos diferenciada**: cualquier usuario podrá **consultar (leer)** el contenido público de la comunidad, pero las facultades de **modificación y eliminación** estarán restringidas exclusivamente al **propietario** o autor de dicho recurso.
- **Visibilidad pública**: por defecto, el contenido generado en la plataforma está diseñado para ser **accesible y compartido** con el resto de los usuarios de la comunidad.

2.2.2. Contenido de eventos

Los eventos no son solo texto; deben soportar **contenido enriquecido**:

- **Información básica**: título, descripción textual y fechas.
- **Adjuntos**: capacidad para asociar **archivos, imágenes y referencias a ubicaciones** (mapas) a un evento específico.

2.2.3. Funcionalidad de búsqueda

Se requiere un **motor de búsqueda** que permita localizar calendarios y eventos mediante **filtros individuales o combinados**:

- **Texto libre**: búsqueda por **palabras clave** en títulos o descripciones.
- **Temporalidad**: filtrado por **rango de fechas** (fecha de creación o fecha del evento).
- **Autoría**: búsqueda de calendarios creados por un **organizador (usuario) específico**.

2.3. Requisitos de integración específica

2.3.1. Visualización de mapas

Integración con el servicio de mapas **OpenStreetMap** para dotar de contexto espacial a los eventos:

- **Visualización**: mostrar **mapas interactivos** con la ubicación exacta de los eventos.

- **Optimización y caching (Geocoding):** en el servicio de geocodificación (convertir direcciones en coordenadas), es obligatorio implementar un **sistema de caching** para almacenar los resultados y **minimizar el consumo de cuotas** de la API externa.

2.3.2. Visualización y almacenamiento de imágenes en la nube

El sistema debe gestionar la **subida y almacenamiento** de imágenes de forma **eficiente**:

- **Carga de archivos:** las imágenes deben subirse desde el dispositivo del usuario (**upload local**), no mediante enlaces externos (URLs).
- **Almacenamiento cloud:** los archivos físicos **no se guardarán en la base de datos** ni en el sistema de archivos local del servidor web, sino en un servicio de almacenamiento en la nube externo (**AWS S3**)
- **Visualización:** la interfaz debe recuperar y mostrar estas imágenes almacenadas en el contexto de los eventos.

2.3.3. Comentarios y notificaciones

Fomento de la interacción entre usuarios y sistema de alertas:

- **Comentarios:** los usuarios pueden escribir comentarios en eventos o calendarios **creados por otros**.
- **Alertas al organizador:** el creador de un evento recibirá una **notificación automática** cuando alguien comente en su publicación.
- **Preferencias de notificación:** el usuario debe poder configurar el canal de recepción de estas alertas:
 - **Email:** envío inmediato al correo electrónico.
 - **In-App:** visualización la próxima vez que inicie sesión en la plataforma.

3. Tecnologías usadas

3.1. Backend

3.1.1. Lenguaje

- **Python:** lenguaje principal del backend, utilizado para implementar la lógica de negocio y los servicios de la aplicación.

3.1.2. Framework

- **FastAPI:** framework web moderno para construir APIs REST rápidas y tipadas, con validación automática de datos, generación de documentación interactiva (OpenAPI/Swagger) y soporte nativo para programación asíncrona.

3.1.3. Contenedores

- **Docker:** cada servicio se ejecuta en su propio contenedor. Se define un Dockerfile por servicio para garantizar entornos reproducibles, facilitar el despliegue en distintos entornos y aislar dependencias. Orquestación usando docker-compose

3.1.4. Gestión de paquetes

- **pip:** gestor de paquetes de Python utilizado para instalar y versionar las dependencias del backend (frameworks, librerías auxiliares, clientes de APIs, etc.).

3.1.5. Acceso a APIs externas

- **httpx:** cliente HTTP asíncrono utilizado desde el backend para consumir servicios externos (por ejemplo, la API de Nominatim de OpenStreetMap), con soporte para timeouts y manejo de errores.

3.1.6. Base de datos

- **MongoDB Atlas:** base de datos NoSQL en la nube. Se utiliza para almacenar usuarios, calendarios, eventos, notificaciones y la caché de geocodificación, aprovechando esquemas flexibles y consultas sobre documentos anidados.

3.2. Frontend

3.2.1. Lenguaje

- **TypeScript:** lenguaje tipado para el frontend, que mejora la mantenibilidad del código, aporta autocompletado más rico y ayuda a detectar errores en tiempo de desarrollo.

3.2.2. Framework

- **React:** framework principal de UI para construir componentes reutilizables y vistas reactivas, gestionando el estado de forma declarativa y facilitando la composición de la interfaz.

3.2.3. Build / bundler

- **Vite:** herramienta de build y bundler moderno con recarga en caliente muy rápida durante el desarrollo y generación de bundles optimizados para producción.

3.2.4. Estilos

- **Tailwind CSS:** framework de utilidades CSS para diseño responsivo, que permite maquetar la interfaz mediante clases atómicas y mantener un diseño visual coherente.
- **PostCSS:** procesador de CSS en la cadena de build (autoprefijos, compatibilidad entre navegadores y optimización de estilos para producción).

3.2.5. Cliente HTTP

- **Axios:** cliente HTTP en el frontend para realizar peticiones REST al backend, con interceptores, configuración centralizada de cabeceras y manejo homogéneo de errores.

3.2.6. Gestión de paquetes

- **npm:** gestor clásico de paquetes de Node.js utilizado para instalar dependencias del frontend, ejecutar scripts y generar el `package-lock.json`.
- **pnpm:** gestor alternativo de paquetes, más rápido y eficiente en espacio gracias a su almacén compartido, útil cuando se trabaja con múltiples proyectos o monorepos.

3.3. Integraciones externas

3.3.1. Almacenamiento

- **Amazon S3:** almacenamiento de objetos para archivos estáticos e imágenes asociadas a eventos o calendarios, con control de acceso y posibilidad de servir el contenido a través de CDN.

3.3.2. Calendarios

- **Google Calendar:** importación y sincronización de calendarios desde cuentas de Google, para integrar agendas externas en la aplicación.
- **TeamUp Calendar:** importación de calendarios de TeamUp, permitiendo centralizar eventos procedentes de esta plataforma.

Cabe destacar que la implementación completa de estas integraciones todavía está en desarrollo y se terminará en la práctica 4 de la asignatura..

3.3.3. Geolocalización

- **OpenStreetMap (Nominatim API):** servicio de geocodificación y mapas. Se utiliza para convertir direcciones en coordenadas y viceversa, con un sistema de caché de geocodificación (módulo `geocode_cache.py`) almacenado en MongoDB para reducir latencia y llamadas repetidas.

3.3.4. Correo electrónico

- **SendGrid:** proveedor de email para el envío de correos transaccionales y notificaciones (por ejemplo, resúmenes, avisos de eventos o cambios en calendarios).

4. Base de datos

4.1. Cambios en el diseño

El **robusto diseño previo de la base de datos** ha permitido que los cambios necesarios para esta práctica sean **mínimos**. La estructura principal basada en las colecciones `users`, `calendars`, `events` y `notifications` se mantiene sin alteraciones, así como la combinación de **desnormalización** (datos duplicados para lecturas eficientes) y **referenciación mediante identificadores** para expresar relaciones entre entidades. Gracias a este diseño inicial, ha sido posible incorporar nuevas funcionalidades sin romper el modelo anterior ni requerir migraciones complejas.

4.1.1. Nueva colección: caché de geocodificación

Se ha añadido una nueva colección llamada `geocode_cache`, destinada a almacenar en caché las respuestas de la API de geocodificación de OpenStreetMap (Nominatim). El objetivo principal es **reducir el número de llamadas externas**, mejorar la **latencia** de las peticiones de geolocalización y **respetar los límites de uso** del servicio de terceros.

Cada documento de esta colección guarda, entre otros campos, el **tipo de consulta** (`geocode`, `reverse`, `search`), los **parámetros originales** (`query_params`), la **respuesta completa** (`response_data`), métricas de uso (`hit_count`, `last_accessed`) y una **fecha de expiración** (`expires_at`).

Para evitar que la caché crezca de forma indefinida, se ha creado un **TTL index** sobre el campo `expires_at`, de manera que **MongoDB elimina automáticamente** los registros caducados tras **siete días**. De este modo, la caché se mantiene limpia sin necesidad de tareas de mantenimiento adicionales en la aplicación.

4.1.2. Ampliación de documentos existentes

Además de la nueva colección, se han realizado pequeñas **ampliaciones en los documentos ya existentes**, siempre siguiendo la filosofía de no romper el diseño previo:

- En la colección `users` se ha extendido el subdocumento `notification_preferences` con nuevos campos (por ejemplo, la **frecuencia de notificaciones**), lo que permite soportar funcionalidades como el resumen diario sin afectar a los datos ya guardados.
- En la colección `events` se ha enriquecido el subdocumento `location` con información adicional sobre el **proveedor de mapas** (`map_provider`), permitiendo distinguir entre ubicaciones procedentes de Google Maps u OpenStreetMap, con el fin de facilitar posibles integraciones futuras.
- En las colecciones principales se mantienen y refuerzan los **índices e invariantes** existentes (búsqueda por texto, jerarquía de calendarios, índices por rangos de fechas, etc.), añadiendo nuevos índices cuando ha sido necesario.

Es importante destacar que **no se ha eliminado ni modificado ningún campo previo**, sino que únicamente se han **añadido nuevos atributos y una nueva colección**. Como consecuencia, **toda la funcionalidad de la práctica anterior permanece intacta** y las aplicaciones que ya consumían el modelo original siguen funcionando sin cambios, beneficiándose al mismo tiempo de las nuevas capacidades de geolocalización y notificaciones avanzadas.

4.2. Instrucciones de acceso

Nuestra base de datos está desplegada utilizando **MongoDB Atlas**. Esto significa que no está alojada en un servidor local (como localhost), sino **en la nube**. La URI del despliegue de la base de datos es la siguiente:

```
mongodb+srv://<db_username>:<db_password>@basmaticcluster.revmyok.mongodb.net/basmati?retryWrites=true&w=majority
```

Esta cadena no contiene la contraseña real ni el usuario por cuestiones de seguridad. La URI completa se configura mediante la variable de entorno **MONGO_URI** (en el fichero .env de la aplicación).

5. Microservicios

5.1. Versionado de los microservicios

El proyecto **Basmati** implementa un sistema de **versionado de API basado en URL** con la estrategia **/v1/** y **/v2/** en la ruta. Este enfoque sigue las mejores prácticas de diseño de **APIs RESTful**, garantizando la estabilidad del sistema mientras se introducen nuevas funcionalidades.

5.1.1. Arquitectura del versionado

5.1.1.1. Organización del sistema

Para mantener la independencia entre versiones, cada microservicio (**Calendar Service** y **Event Service**) organiza su lógica interna de forma que los componentes de la **API Versión 1** y la **API Versión 2** residan en espacios separados. Esto permite que los clientes **reutilicen** los endpoints de **V1** que no han cambiado, mientras acceden a **V2** solo para las nuevas funcionalidades, evitando duplicación innecesaria de código en el backend.

5.1.1.2. Implementación de rutas

El acceso a los servicios se gestiona mediante routers independientes. Al montar el sistema, se asignan prefijos específicos que determinan la versión a la que el cliente está accediendo. De esta forma, las URLs de consulta quedan claramente diferenciadas para el usuario final (por ejemplo, diferenciando entre consultas generales de la versión base y las avanzadas de la versión nueva).

5.1.2. Patrón de diseño: Abstract Factory

Para la gestión de las diferentes versiones, se ha implementado el patrón de diseño **Abstract Factory**. Este patrón permite manejar la creación de objetos y servicios sin especificar sus clases concretas de antemano.

- **Fábricas por versión:** existen fábricas específicas para la **V1** y la **V2**. Cada una se encarga de instanciar los repositorios y servicios de lógica de negocio adecuados para su versión.
- **Registro centralizado:** el sistema utiliza un registro de fábricas que mapea cada servicio con su versión correspondiente, facilitando la escalabilidad si en el futuro se requieren versiones adicionales (**V3**, **V4**, etc.).
- **Inyección de dependencias:** cada ruta de la API utiliza una función de dependencia específica. Esto asegura que, dependiendo de si el usuario accede a la ruta de la **V1** o la **V2**, el sistema le entregue automáticamente el servicio con la lógica de negocio correcta.

5.1.3. Filosofía de desarrollo incremental

El proyecto sigue una filosofía de desarrollo eficiente: **solo se incluye en la Versión 2 aquello que es nuevo o que ha sido modificado**.

- Los componentes de la **V2** contienen exclusivamente los puntos de acceso que presentan cambios respecto a la **V1**.
- Si un punto de acceso no ha sufrido modificaciones, se mantiene la referencia a la versión original, optimizando el mantenimiento del código.

5.2. Cambios en los microservicios

5.2.1. Calendar Service

Para esta entrega se desarrollaron nuevos endpoints y se mejoraron los existentes en la versión 2 (V2) para ofrecer mayor flexibilidad y funcionalidad.

5.2.1.1. GET /v2/calendars – Obtener todos los calendarios

Obtiene una lista de todos los calendarios del sistema.

- **Parámetros:**
 - limit (opcional): Número máximo de calendarios a devolver (por defecto 200, máximo 1000).
- **Respuesta:** Lista de objetos CalendarResponse.

5.2.1.2. DELETE /v2/calendars/{calendar_id}/recursive – Eliminar calendario recursivamente

Elimina un calendario junto con toda su jerarquía (subcalendarios y eventos asociados).

- **Respuesta:** Objeto JSON confirmando la eliminación.

5.2.1.3. GET /v2/calendars/search/by-text – Buscar por texto

Realiza una búsqueda de calendarios basada en texto

- **Parámetros:**
 - query (requerido, query): Texto a buscar.
- **Respuesta:** Lista de objetos CalendarResponse.

5.2.1.4. GET /v2/calendars/search/by-creator-name – Buscar por nombre del creador

Busca calendarios basándose en el nombre del creador.

- **Parámetros:**
 - name (requerido, query): Nombre del creador.
- **Respuesta:** Lista de objetos CalendarResponse.

5.2.1.5. POST /v2/calendars/{calendar_id}/comments – Agregar comentario a un calendario

Agregar comentario a un calendario público.

- **Parámetros:**
 - calendar_id (requerido, path): ID del calendario.
 - current_user_id (requerido, query): ID del usuario actual.
- **Respuesta:** Objeto CalendarComment creado.

5.2.2. Event Service

Para esta entrega, se desarrollaron dos nuevos endpoints y se modificó uno existente.

5.2.2.1. GET /v2/events – Obtener todos los eventos

Obtener una lista de todos los eventos del sistema.

- **Parámetros:**
 - limit (opcional): Número máximo de eventos a devolver (por defecto 200, máximo 1000).
- **Respuesta:** Lista de objetos EventResponse.

5.2.2.2. DELETE /v2/events/by-calendar/{calendar_id} – Eliminar eventos de un calendario

Elimina todos los eventos asociados a un calendario específico. Utilizado para eliminación en cascada.

- **Parámetros:**
 - calendar_id (requerido, path): ID del calendario.
- **Respuesta:** Objeto JSON con mensaje de confirmación y conteo de eliminados.

5.2.2.3. GET /v2/events/search/by-date-range – Buscar eventos por rango de fechas

Busca eventos dentro de un rango de fechas.

- **Parámetros:**
 - start (requerido, query): Fecha inicio ISO 8601.
 - end (requerido, query): Fecha fin ISO 8601.
 - calendar_id (opcional, query): ID del calendario para filtrar (**Nuevo en V2**).
- **Respuesta:** Lista de objetos EventResponse.

5.2.2.4. POST /v2/events/{event_id}/comments – Agregar comentario

Agrega un comentario a un evento. En V2 el backend resuelve automáticamente el display_name del usuario.

- **Parámetros:**
 - event_id (requerido, path): ID del evento.
 - comment (requerido, body): JSON con user_id y text.
- **Respuesta:** Objeto EventComment creado.

5.2.2.5. GET /v2/events/search/by-text - Búsqueda full-text en eventos

Realiza una búsqueda de texto completo en eventos. Busca en título, descripción y ubicación.

- **Parámetros:**
 - query (requerido, query): Término de búsqueda.
- **Respuesta:** Lista de objetos EventResponse que coinciden con la búsqueda.

5.2.2.6. GET /v2/events/search/by-calendar-title - Buscar eventos por título del calendario

Busca eventos por título del calendario (campo desnormalizado).

- **Parámetros:**
 - calendar_title (requerido, query): Título o parte del título del calendario.
- **Respuesta:** Lista de objetos EventResponse del calendario con ese título.

5.2.2.7. GET /v2/events/search/by-location - Buscar eventos por ubicación

Busca eventos por dirección o nombre del lugar.

- **Parámetros:**
 - location_query (requerido, query): término de búsqueda para la ubicación (busca en address y place_name).
- **Respuesta:** Lista de objetos EventResponse en esa ubicación.

5.2.2.8. GET /v2/events/search/advanced - Búsqueda avanzada de eventos

Realiza una búsqueda avanzada combinando múltiples criterios (lógica AND). Busca por título, organizador (calendario) y palabras clave (descripción).

- **Parámetros:**
 - title (opcional, query): Título del evento.
 - organizer (opcional, query): Título del calendario (organizador).
 - keywords (opcional, query): Palabras clave a buscar en la descripción.
- **Respuesta:** Lista de objetos EventResponse que coinciden con todos los criterios especificados.

5.2.3. Notification Service

El servicio de notificaciones mantiene su estabilidad en la versión actual. No se han introducido cambios en la API pública para la versión 2 en esta entrega, continuando con el uso de los endpoints V1 definidos en api/v1. La infraestructura subyacente soporta las nuevas características de frecuencia introducidas en el User Service.

5.2.4. Integration Service

Se han añadido múltiples integraciones y servicios auxiliares en la versión 2, incluyendo geocodificación, almacenamiento S3 y gestión de correos.

5.2.4.1. POST /v2/integrations/imports/google – Importar desde Google Calendar

Importa calendarios desde Google Calendar.

- **Parámetros:**
 - import_request (requerido, body): Token y IDs de calendarios.
- **Respuesta:** ImportResponse con los resultados.

5.2.4.2. POST /v2/integrations/imports/teamup – Importar desde Teamup

Importa calendarios desde Teamup.

- **Parámetros:**
 - import_request (requerido, body): Datos de importación (user_external_id, calendar_keys).
- **Respuesta:** ImportResponse con los resultados.

5.2.4.3. GET /v2/integrations/osm/geocode – Geocodificar dirección

Convierte una dirección en coordenadas usando OpenStreetMap.

- **Parámetros:**
 - address (requerido, query): Dirección a buscar.
 - limit (opcional, query): Máximo de resultados.
- **Respuesta:** GeocodeResponse con lista de ubicaciones.

5.2.4.4. GET /v2/integrations/osm/reverse-geocode – Geocodificación inversa

Convierte coordenadas en una dirección legible.

- **Parámetros:**
 - latitude (requerido, query): Latitud.
 - longitude (requerido, query): Longitud.
- **Respuesta:** ReverseGeocodeResponse con la ubicación.

5.2.4.5. GET /v2/integrations/osm/search-places – Buscar lugares

Busca lugares de interés por nombre o tipo.

- **Parámetros:**
 - query (requerido, query): Término de búsqueda.
 - near_latitude / near_longitude (opcional, query): Coordenadas para priorizar cercanía.
- **Respuesta:** GeocodeResponse.

5.2.4.6. GET /v2/integrations/osm/cache/stats - Estadísticas del caché de geocodificación

Obtiene estadísticas del sistema de caché de geocodificación (total de entradas, hits, estadísticas por tipo de consulta).

- **Parámetros:** Ninguno.
- **Respuesta:** Objeto JSON con estadísticas del caché.

5.2.4.7. DELETE /v2/integrations/osm/cache - Limpiar caché de geocodificación

Elimina todas las entradas del caché de geocodificación.

- **Parámetros:** Ninguno.
- **Respuesta:** Objeto JSON con número de entradas eliminadas.

5.2.4.8. POST /v2/integrations/s3/upload-direct – Subir imagen directa

Sube una imagen con compresión automática a S3.

- **Parámetros:**
 - file (requerido, form-data): Archivo de imagen.
 - folder (opcional, query): Carpeta destino.
 - compress (opcional, query): Activar compresión.
- **Respuesta:** ImageMetadata de la imagen subida.

5.2.4.9. POST /v2/integrations/s3/upload – Obtener URL presigned

Genera una URL presignada para subir una imagen directamente a S3 desde el cliente, sin pasar el archivo por el backend.

- **Parámetros:**
 - filename (requerido, body): Nombre del archivo.
 - content_type (opcional, body): Tipo de archivo (MIME).
- **Respuesta:** JSON con presigned_url y campos de formulario.

5.2.4.10. GET /v2/integrations/s3/images - Listar imágenes

Lista las imágenes almacenadas en S3.

- **Parámetros:**
 - prefix (opcional, query): Prefijo/carpeta para filtrar (ej: 'events', 'avatars').
 - max_results (opcional, query): Número máximo de resultados (por defecto 100, máximo 1000).
- **Respuesta:** ImageListResponse con lista de imágenes y metadatos.

5.2.4.11. GET /v2/integrations/s3/images/{image_key} - Obtener metadatos de imagen

Obtiene los metadatos de una imagen específica en S3.

- **Parámetros:**
 - image_key (requerido, path): Key de la imagen en S3 (puede incluir carpeta).
- **Respuesta:** ImageMetadata con metadatos de la imagen.

5.2.4.12. DELETE /v2/integrations/s3/images/{image_key} - Eliminar imagen

Elimina una imagen del bucket de S3.

- **Parámetros:**
 - image_key (requerido, path): Key de la imagen a eliminar.
- **Respuesta:** ImageDeleteResponse con resultado de la eliminación.

5.2.4.13. GET /v2/integrations/s3/download/{image_key} - Obtener URL de descarga

Genera una URL presigned temporal para descargar una imagen.

- **Parámetros:**
 - image_key (requerido, path): Key de la imagen en S3.
 - expiration (opcional, query): Tiempo de expiración en segundos (por defecto 3600, máximo 43200).
- **Respuesta:** ImageDownloadResponse con URL temporal de descarga.

5.2.4.14. POST /v2/integrations/s3/images/bulk-delete - Eliminar múltiples imágenes

Elimina múltiples imágenes del bucket de S3 en una sola operación (máximo 1000 imágenes por llamada).

- **Parámetros:**
 - request (requerido, body): Lista de keys de imágenes a eliminar.
- **Respuesta:** BulkDeleteResponse con contador de eliminaciones y errores.

5.2.4.15. GET /v2/integrations/s3/images/{image_key}/exists - Verificar si imagen existe

Verifica si una imagen existe en el bucket de S3.

- **Parámetros:**
 - image_key (requerido, path): Key de la imagen a verificar.
- **Respuesta:** Objeto JSON con exists (boolean) y key.

5.2.4.16. POST /v2/integrations/email/send – Enviar correo

Envía un correo electrónico a través de SendGrid.

- **Parámetros:**
 - email_request (requerido, body): Destinatario, asunto y contenido.
- **Respuesta:** EmailResponse con estado del envío.

5.2.4.17. POST /v2/integrations/email/send-digest – Enviar resumen diario

Envía manualmente el correo de resumen diario de notificaciones. Necesita OAuth para funcionar bien. La implementación actual está mockeada.

- **Parámetros:**
 - digest_request (requerido, body): Datos del usuario y lista de notificaciones.
- **Respuesta:** EmailResponse con estado del envío.

5.2.4.18. POST /v2/integrations/email/send-comment-notification – Enviar notificación de comentario

Envía un correo electrónico notificando sobre un nuevo comentario recibido.

- **Parámetros:**
 - notification_request (requerido, body): Detalles del comentario y destinatario.
- **Respuesta:** EmailResponse.

5.2.4.19. POST /v2/integrations/digest/send-all – Enviar digest masivo

Ejecuta el proceso batch para enviar el resumen diario a todos los usuarios que tienen configurada la frecuencia 'daily'.

- **Respuesta:** JSON con el número de correos enviados.

5.2.4.20. POST /v2/integrations/digest/send-user – Enviar digest a usuario

Fuerza el envío del resumen diario para un usuario específico.

- **Parámetros:**
 - user_external_id (requerido, query): ID del usuario.
- **Respuesta:** Estado del envío.

5.2.4.21. GET /v2/integrations/digest/preview/{user_external_id} - Vista previa del resumen diario

Genera una vista previa del digest sin enviarlo. Útil para desarrollo y pruebas.

- **Parámetros:**
 - user_external_id (requerido, path): ID externo del usuario.
- **Respuesta:** DigestPreviewResponse con vista previa del digest.

5.2.5. User Service

La versión 2 introduce soporte para preferencias de notificación avanzadas y nuevos filtros.

5.2.5.1. GET /v2/users/by-external-id/{external_id} – Obtener por External ID

Obtiene un usuario utilizando su ID externo (OAuth).

- **Parámetros:**
 - external_id (requerido, path): ID externo del usuario.
- **Respuesta:** UserResponseV2 con datos del usuario.

5.2.5.2. GET /v2/users – Listar usuarios

Lista usuarios permitiendo filtrar por preferencias de notificación.

- **Parámetros:**
 - notification_frequency (opcional, query): 'instant' o 'daily'.
 - email_notifications (opcional, query): true/false.
 - limit (opcional, query): Límite de resultados.
- **Respuesta:** Objeto JSON con lista de usuarios y conteo.

5.2.5.3. GET /v2/users/{user_id}/notification-preferences – Obtener preferencias

Obtiene las preferencias de notificación de un usuario, incluyendo frecuencia.

- **Parámetros:**
 - user_id (requerido, path): ID del usuario.
- **Respuesta:** NotificationPreferencesSchemaV2.

5.2.5.4. PUT /v2/users/{user_id}/notification-preferences – Actualizar preferencias

Actualiza las preferencias de notificación, incluyendo la configuración de frecuencia ('instant' o 'daily').

- **Parámetros:**
 - user_id (requerido, path): ID del usuario.
 - preferences (requerido, body): Objeto con nuevas preferencias.
- **Respuesta:** NotificationPreferencesSchemaV2 actualizadas.

5.2.5.5. GET /v2/users/{user_id} – Obtener usuario por ID

Obtiene los datos de un usuario específico utilizando el esquema V2 (incluye preferencias de frecuencia).

- **Parámetros:**
 - user_id (requerido, path): ID interno del usuario.
- **Respuesta:** UserResponseV2.

5.2.5.6. PUT /v2/users/{user_id} – Actualizar usuario

Actualiza la información de un usuario existente.

- **Parámetros:**
 - user_id (requerido, path): ID del usuario.
 - user_update (requerido, body): Objeto con los campos a modificar.
- **Respuesta:** UserResponseV2 actualizado.

5.2.5.7. POST /v2/users/seed-dev-users – Crear usuarios de prueba

Endpoint de utilidad para poblar la base de datos con usuarios de desarrollo predefinidos.

- **Respuesta:** JSON con la lista de IDs de usuarios creados.

6. Frontend

6.1. Funcionalidad

6.1.1. Gestión de sesión y usuarios

- **Login de usuarios:** sistema de inicio de sesión basado en *external_id* (IDs externos), con sugerencias de usuarios de desarrollo para facilitar las pruebas. Actualmente está hardcodeado porque no hay implementación de OAuth (siguiente práctica).
- **Gestión de perfil:** visualización y edición de datos del usuario (nombre, ID, correo).
- **Preferencias de notificación:** configuración granular donde el usuario decide el canal de recepción: correo electrónico o dentro de la app y frecuencia de la notificación para el daily digest (por implementar, pues necesita OAuth).

6.1.2. Gestión de calendarios (Dashboard)

- **Vistas múltiples:** el dashboard permite visualizar la agenda en cuatro formatos distintos:
 - Vista de **mes**.
 - Vista de **año**.
 - Vista de **semana**.
 - Vista de **día**.
- **CRUD de calendarios:** creación, lectura, modificación y eliminación de calendarios propios.
- **Importación:** funcionalidad para importar calendarios externos (TeamUp y Google Calendar). No totalmente funcional pues necesita OAuth. Se terminará de arreglar en la siguiente práctica.
- **Sidebar responsiva:** menú lateral plegable para la gestión de "Mis calendarios" y "Otros calendarios".

6.1.3. Gestión de eventos

- **Creación y edición completa:** los eventos incluyen título, descripción y fechas, soportando contenido enriquecido.
- **Adjuntos multimedia (S3):** subida de imágenes directamente desde el dispositivo (drag & drop o clic), almacenadas en AWS S3.
- **Geolocalización (mapas):** integración de mapas interactivos (OpenStreetMap) para visualizar la ubicación exacta de los eventos.
- **Sistema de comentarios:** posibilidad de añadir comentarios en los eventos y calendarios, fomentando la interacción social.

6.1.4. Búsqueda global y filtrado

Se ha implementado un motor de búsqueda avanzado accesible desde la interfaz:

- **Búsqueda por texto libre:** localiza eventos o calendarios por palabras clave.
- **Filtros avanzados:**
 - **Temporal:** filtrado por rango de fechas (desde/hasta).
 - **Autoría:** filtrado por nombre del organizador o creador.

6.1.5. Aspectos técnicos de la interfaz (UI/UX)

- **Diseño responsivo (Mobile First):** La interfaz se adapta a móviles (menú tipo *drawer*), tablets y escritorio .

- **Accesibilidad (WCAG 2.1 AA):** navegación completa por teclado, etiquetas ARIA para lectores de pantalla y alto contraste visual para usuarios con visión reducida.
- **Feedback visual:** uso de colores semánticos para indicar éxito, error o información (ej. alertas rojas para acciones destructivas) .

6.2. Diseño

6.2.1. Accesibilidad

El desarrollo de la interfaz de usuario se ha regido por las directrices de accesibilidad web **WCAG 2.1 nivel AA**, asegurando que la aplicación sea utilizable por el mayor número posible de personas, independientemente de sus capacidades. Para lograr este objetivo, se han implementado las siguientes medidas técnicas:

- **HTML semántico:** se ha priorizado el uso de etiquetas HTML estándar con significado semántico (<nav>, <aside>, <main>, <button>) en lugar de elementos genéricos (<div>), facilitando la interpretación de la estructura por parte de los **lectores de pantalla**.
- **Etiquetas ARIA:** se han incorporado atributos aria-label, aria-expanded y role en componentes interactivos complejos (como el menú lateral desplegable y los modales) para proporcionar contexto adicional a las **tecnologías de asistencia**.
- **Navegación por teclado:** toda la interfaz es operable mediante **tabulación**. Se han gestionado los **estados de foco (focus ring)** para que sean claramente visibles, permitiendo al usuario saber siempre en qué elemento interactivo se encuentra.
- **Contraste de color:** el estilo visual elegido garantiza **un ratio de contraste alto** entre el texto y el fondo, superando el umbral de **4.5:1** requerido para texto normal, lo que mejora la legibilidad para usuarios con visión reducida.

A continuación, se adjunta una captura del análisis de accesibilidad realizado con la extensión **WAVE (Web Accessibility Evaluation Tool)**, donde se verifica la ausencia de errores de contraste y la correcta estructura semántica:

The screenshot displays the WAVE extension interface on a web browser. On the left, the WAVE sidebar shows a summary of accessibility issues: 0 Errors, 0 Contrast Errors, 9 Alerts, 2 Features, 14 Structure errors, and 311 ARIA issues. A message states: "Congratulations! No errors were detected! Manual testing is still necessary to ensure compliance and optimal accessibility. AIM Score: 9.8 out of 10". Below this, it lists 9 Alerts, with one specific alert: "9 Redundant title text".

The main content area shows a calendar application for December 2025. The calendar has a header with navigation buttons and a search bar. The days of the week are listed: LUNES, MARTES, MIÉRCOLES, JUEVES, VIERNES, SÁBADO, DOMINGO. The calendar grid shows dates from 1 to 31. Several events are visible: "prueba pierna" on Tuesday the 2nd, "Curso 2026" on Wednesday the 3rd, "QUEDAD..." on Thursday the 4th, "PRUEBA 2" on Friday the 5th, "FUNCIONA" on Saturday the 6th, "Capturas de instalación de wordpress" on Sunday the 7th, and "yujui" on Monday the 8th. A "Tip del día" (Tip of the day) is shown at the bottom: "Revisa si estás dejando hueco para imprevistos agradables."

La aplicación implementa un diseño adaptable (**Responsive Web Design**) siguiendo la metodología **Mobile First**. La interfaz se ajusta fluidamente a diferentes resoluciones mediante el uso de **Media Queries** y el sistema de rejilla flexible de **Tailwind CSS**:

- **Dispositivos móviles:** el menú de navegación se comporta como un panel lateral oculto (**tipo drawer**) que se superpone al contenido para maximizar el espacio útil. La vista de calendario se simplifica para mostrar **listas o días individuales**.
- **Tablets y portátiles:** el diseño aprovecha el mayor ancho para **fixar la barra lateral** y expandir la rejilla del calendario.
- **Escritorio (Desktop):** se despliega la **interfaz completa**, permitiendo la visualización detallada del **mes completo** y paneles de información complementaria simultáneamente.

6.2.2. Estilo

La identidad visual de Basmati se basa en el **Neo-Brutalismo**, una tendencia de diseño moderna que prioriza la funcionalidad, la crudeza visual y el **alto contraste**. Este estilo no solo aporta una estética diferenciadora y atrevida, sino que refuerza la usabilidad al delimitar claramente los componentes interactivos.

6.2.2.1. Características principales

- **Bordes y sombras:** uso de **bordes gruesos (3px)** de color negro sólido y sombras duras (sin difuminado) que crean una sensación de profundidad y tacto "físico".
- **Tipografía:** fuentes **Sans-serif en negrita** para jerarquizar la información de manera clara y directa.

6.2.2.2. Paleta de colores

Se ha definido una paleta de colores vibrante pero limitada para mantener la consistencia:

- **Color principal:** **#EBBE4D (Basmati Yellow)** - Usado para acciones principales y elementos destacados.
- **Fondo:** **#FFFAEB (Basmati White/Cream)** - Un tono crema suave que reduce la fatiga visual comparado con el blanco puro.
- **Texto y bordes:** **#1A1A1A (Basmati Black)** - Negro suave para garantizar máximo contraste y legibilidad.
- Colores semánticos:
 - **#5496FF (Azul):** información y enlaces.
 - **#FF6B6B (Rojo):** acciones destructivas y errores.
 - **#4ECDC4 (Verde):** confirmaciones y estados de éxito.

6.3. Integración de servicios externos

6.3.1. Visualización de mapas – OpenStreetMap

OpenStreetMap (OSM) es un proyecto colaborativo de mapeo global de **código abierto**, que proporciona datos geográficos libres y editables bajo una **licencia abierta**. Fundado en 2004, opera como una base de datos colectiva donde voluntarios contribuyen con información sobre carreteras, edificios, puntos de interés y otros elementos cartográficos. A diferencia de servicios comerciales como Google Maps, OSM no impone restricciones de uso **ni requiere claves de API** para consultas básicas, lo que lo hace ideal para integraciones en aplicaciones de código abierto o con presupuestos limitados.

6.3.1.1. Relación con el servicio de integración

En el contexto del servicio de integración de la aplicación **Basmati**, OSM se utiliza como **fuentes primaria para operaciones de geocodificación**. El servicio interactúa con la **API de OSM** (específicamente, servicios como Nominatim para **geocodificación inversa y directa**) para convertir direcciones en coordenadas geográficas o viceversa. Esta integración permite al servicio proporcionar funcionalidades de mapeo y localización sin depender de proveedores propietarios, alineándose con principios de accesibilidad y sostenibilidad.

6.3.1.2. Funcionalidades implementadas

- **Geocodificación de direcciones:** endpoint que acepta direcciones textuales y devuelve listas de ubicaciones con coordenadas, priorizando resultados relevantes.
- **Geocodificación inversa:** convierte coordenadas en direcciones formateadas, útil para contextualizar posiciones geográficas.
- **Búsqueda de lugares:** permite búsquedas por nombre o tipo, con opción de priorizar resultados cercanos a coordenadas específicas.
- **Gestión del caché:** endpoints para consultar estadísticas del caché y limpiarlo, facilitando el mantenimiento y la actualización de datos.

Esta implementación asegura una integración robusta, con manejo de errores detallado y documentación automática vía **OpenAPI**, permitiendo una interacción eficiente con OSM mientras se mantiene la escalabilidad del servicio.

6.3.2. Almacenamiento de imágenes – AWS S3

El sistema de almacenamiento **S3 (Simple Storage Service) de Amazon Web Services (AWS)** se integra en el Servicio de Integración de la aplicación Basmati como solución para la gestión de imágenes. Este servicio permite el almacenamiento, recuperación y manipulación de archivos multimedia de manera escalable y eficiente, utilizando un bucket dedicado en la nube de AWS. La implementación se basa en la biblioteca boto3 para interactuar con la API de S3, ofreciendo operaciones **CRUD completas** con optimizaciones como **compresión automática** para reducir costos de almacenamiento y ancho de banda.

6.3.2.1. Relación con el servicio de integración

En el contexto del Servicio de Integración, S3 actúa como **repositorio centralizado** para imágenes asociadas a eventos, avatares de usuarios, calendarios y adjuntos generales. El servicio expone **endpoints FastAPI versionados (v2)** que facilitan la subida directa o mediante **URLs presigned**, la compresión automática de imágenes y la gestión de metadatos. Esta integración asegura que las imágenes sean accesibles de forma segura y optimizada, complementando otras funcionalidades como la geocodificación.

6.3.2.2. Funcionalidades implementadas

- **Subida de imágenes:** soporta dos métodos: subida directa con compresión automática en el backend, y generación de **URLs presigned** para subida directa desde el cliente, mejorando el rendimiento al evitar tránsito innecesario de datos.
- **Compresión y optimización:** las imágenes se convierten automáticamente a formato **WebP** para máxima eficiencia, redimensionándolas si exceden límites predefinidos (**1920x1080 píxeles**), preservando calidad y reduciendo tamaño hasta en un **30%**.
- **Gestión de metadatos:** permite obtener información detallada de imágenes, incluyendo tamaño, tipo MIME y fecha de modificación, facilitando la administración y verificación.
- **Listado y filtrado:** ofrece la capacidad de listar imágenes por carpetas (e.g., 'events', 'avatars'), con filtros opcionales para optimizar consultas.
- **Descarga segura:** genera **URLs temporales presigned** para descarga, controlando el acceso y expiración para proteger recursos sensibles.
- **Eliminación:** incluye operaciones individuales y masivas para eliminar imágenes, con validaciones para evitar pérdidas accidentales.
- **Utilidades adicionales:** verificación de existencia de imágenes y copia entre ubicaciones, útiles para backups o reorganizaciones.

La implementación utiliza esquemas **Pydantic** para validar entradas y salidas, asegurando **integridad** en operaciones como subida y eliminación. El servicio **S3ImageService** maneja la lógica de negocio, incluyendo **compresión con Pillow** para optimizar formatos, generación de claves únicas con **UUID** para evitar colisiones, y manejo de errores robusto mediante excepciones específicas.

6.3.3. Envío de emails – SendGrid

SendGrid es una plataforma de entrega de **correo electrónico** basada en la **nube**, especializada en el envío **masivo y transaccional** de emails con **alta tasa de entrega** y capacidades avanzadas de **analítica**. Desarrollada por **Twilio**, ofrece **APIs RESTful** para integrar funcionalidades de email en aplicaciones, incluyendo **plantillas personalizables**, seguimiento de aperturas y clics, y gestión de listas de contactos. Su enfoque en la **entregabilidad** la hace ideal para **notificaciones**, marketing y comunicaciones automatizadas en entornos empresariales.

6.3.3.1. Relación con el servicio de integración

En el contexto del **Servicio de Integración** de la aplicación Basmati, **SendGrid** se emplea como **proveedor principal** para el envío de correos electrónicos, facilitando **notificaciones** y comunicaciones con usuarios. La integración permite enviar **emails individuales**, **resúmenes diarios** y **notificaciones de comentarios**, conectando con otros microservicios para enriquecer la experiencia del usuario mediante **alertas oportunas y personalizadas**.

6.3.3.2. Funcionalidades implementadas

La implementación de **SendGrid** en el **Servicio de Integración** se realiza a través de **endpoints FastAPI versionados (v2)**, exclusivos de esta versión. Utiliza esquemas **Pydantic** para validar solicitudes de email, incluyendo destinatarios, asuntos y contenidos. El servicio **EmailServiceV2** maneja la lógica de envío, integrando con la **API de SendGrid** mediante una clave API configurada, con modo de simulación para entornos de desarrollo. Los endpoints soportan **envío asíncrono**, **formateo HTML** con estilos personalizados de Basmati, y **manejo robusto de errores** para garantizar fiabilidad en las siguientes funcionalidades:

- **Envío de correo electrónico individual:** permite enviar un email único a un destinatario específico, con **validación de datos** como dirección de email, asunto y contenido. El servicio configura el **remite**nte desde una cuenta predefinida y maneja errores de envío.
- **Envío de resumen diario de notificaciones:** facilita el envío de un email compilado con un **resumen de notificaciones diarias**, formateado en **HTML con estilos personalizados** de la aplicación. Agrupa múltiples notificaciones en un solo mensaje para optimizar la comunicación.
- **Envío de notificación de nuevo comentario:** envía **alertas automáticas** cuando se añade un comentario a un calendario o evento, incluyendo detalles del evento y el comentario. Utiliza **plantillas específicas** para notificaciones de comentarios, asegurando relevancia y contexto.

6.3.4. Otros servicios

6.3.4.1. Importación de calendarios – TeamUp

Este servicio fue modificado para esta práctica debido a que se encontraron varios **fallos graves** en su implementación anterior.

- Al importar un calendario desde un usuario, este calendario **no se relacionaba** de ninguna manera con este usuario, no se almacenaba el **external_id**.

- Las **claves necesarias** para la importación de los calendarios y de sus correspondientes eventos eran **incorrectos**, por tanto, se modificaron para permitir a los usuarios **importar sus calendarios favoritos** de este servicio externo.

De igual manera, la **implementación** de la importación de calendarios de **TeamUp** **no está pulida ni terminada**. Se terminará en la práctica 4, donde se pide explícitamente que se implemente.

6.3.4.2. Importación de calendarios – Google Calendar

En cuanto a los calendarios de **Google**, al no poder hacer uso de su mecanismo de autenticación **OAuth**, es **imposible el acceso a su API de calendarios**, por tanto, se **prorrogará el desarrollo** de este apartado de *Integration Service* para la siguiente entrega.

7. Instrucciones de despliegue

7.1. Requisitos previos

- **Docker** y **Docker Compose** instalados.
- Conexión a internet (para la base de datos en MongoDB Atlas).

7.2. Configuración

Debe tener un archivo `.env` en la raíz del proyecto con las credenciales necesarias (MongoDB, AWS S3, SendGrid, etc.).

7.3. Despliegue en Docker Compose

Para compilar y levantar todos los servicios por primera vez:

```
docker-compose build --no-cache; docker-compose up
```

Para levantar los servicios en segundo plano (background):

```
docker-compose up -d
```

Para detener todos los servicios:

```
docker-compose down
```

7.4. Acceso al sistema

Una vez levantado, puedes acceder a las distintas partes del sistema:

- **Frontend (Web):** <http://localhost:5173>
- **API Gateway (Documentación):** <http://localhost:8000/docs>